

Reviewer Pack — Kronecker Coefficient Gap in Symmetric Powers of Permanent v...

Entry 3e4b5c9a803c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-11 14:19:39 UTC

Kronecker Coefficient Gap in Symmetric Powers of Permanent vs Determinant

Entry ID: 3e4b5c9a803c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-11 14:19:39 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory of Symmetric Groups (Kronecker coefficients) **Field B** (complexity object): Boolean Circuit Size for Permanent vs Determinant Polynomials

Statement:

For all $n \geq 1$, the minimal Kronecker coefficient in the decomposition of $S^k(\text{perm}_n)$ is exponentially larger than that in $S^k(\text{det}_m)$ for all $m < n^{\{1.5\}}$ and $k = O(\log n)$.

Rationale (proposer's reasoning):

Kronecker coefficients encode the structure of tensor products of symmetric group representations, which are central to GCT's orbit closure separation. A gap in their asymptotic behavior could imply structural differences between permanent and determinant polynomials, potentially leading to circuit size lower bounds.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7f0707a72a92c4e9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def kronecker_coefficient(a, b):
        if a == 0 or b == 0:
            return 1
        if a < b:
            a, b = b, a
        c = [0] * (b + 1)
        c[0] = 1
        for i in range(1, a + 1):
            d = [0] * (b + 1)
            d[0] = 1
            for j in range(1, min(i, b) + 1):
                d[j] = c[j - 1] + c[j]
            for j in range(min(i, b), 0, -1):
                c[j] = (d[j] + d[j - 1]) // i
        return c[b]

    def symmetric_power_kronecker(n, k):
        return kronecker_coefficient(k, n)

    n_max = 40
    min_ratio = float('inf')
    instances_tested = 0

    for n in range(5, n_max + 1):
        k = int(math.log2(n))
        m = math.isqrt(n * n // 3)

        perm_kronecker = symmetric_power_kronecker(n, k)
        det_kronecker = symmetric_power_kronecker(m, k)

        if det_kronecker == 0:
            continue
```

```

    ratio = perm_kronecker / det_kronecker
    min_ratio = min(min_ratio, ratio)
    instances_tested += 1

conjecture_holds = min_ratio > math.exp(1) ** (math.log2(n_max))
counterexample = "" if conjecture_holds else f"n={n_max}, k={k}, perm_kronecker={perm_kronecker}

return {
    "metric_name": "Kronecker Coefficient Ratio",
    "metric_value": min_ratio,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    avg_ratio = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(result["conjecture_holds"] for result in results) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={avg_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_ratio} std=0.0 support_fraction={support_fraction:.2f}")
    else:
        for result in results:
            if not result["conjecture_holds"]:
                counterexample = result["counterexample"]
                first_failing_seed = seed
                break
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ic_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}
TRIAL: {'metric_name': 'Kronecker Coefficient Ratio', 'metric_value': 0.0, 'instances_tested': 4, 'conjecture_holds': True}

```

RESULT: FALSIFIED counterexample="n=40, k=5, perm_kronecker=0, det_kronecker=0" first_failing_seed=9

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The metric definition bug is evident: both Kronecker coefficients are zero, making the ratio undefined (division by zero). The test reports 0.0 instead of rejecting the ratio computation, leading to a false counterexample. This suggests the metric fails when both values are zero, which is a critical edge case.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test's metric incorrectly reports a ratio of 0.0 when both Kronecker coefficients are zero, making the counterexample invalid. This metric flaw undermines the falsification claim. | next: Fix the metric to handle zero coefficients properly and retest with validated edge cases.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111604
2	propose	ollama_remote	qwen3:8b	0	0	119451
3	preregistration	ollama_remote	qwen3:8b	0	0	24118
4	novelty	ollama_remote	qwen3:8b	0	0	20678
5	novelty	ollama_remote	qwen3:8b	0	0	14378
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13300
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8633
8	critic	ollama_remote	qwen3:8b	0	0	29546
9	judge	ollama_remote	qwen3:8b	0	0	16033

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 357741 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3e4b5c9a803c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3e4b5c9a803c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3e4b5c9a803c.tar.gz` (if generated)